SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA1001	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – **Architecture Design Assignment**
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	U.Sanjay	**Reg. No.:**	1920260111
Date:	27-10-2026	**Duration:**	60 minutes

Code of Conduct

I T. Tarun (1924260001) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: T. Tarun

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Online Food Waste Management System

1. Introduction:

The Online Food Waste Management System is an web-based platform designed to monitor food waste levels, consumption, waste quantity, and collection status in residential and institutional food waste networks. Sensors continuously collect waste records and help identify waste accumulation, bin overflow, unusual consumption, and low food waste availability.

The proposed architecture integrates bin collection points and bin bin sensors and collection-point inputs, data communication, real-time processing, alert management, secure storage, and a web dashboard. It reduces manual monitoring, supports timely maintenance, and helps authorities manage food waste resources efficiently.

2. System Requirements Analysis:

2.1 System Objectives

The main objectives of the Online Food Waste Management System are:

- To monitor food waste waste bin levels and food waste continuously.
- To measure food waste collection and collection status using connected collection points and bin bin sensors.
- To detect waste accumulation, waste bin bin overflow, and abnormal food waste.
- To notify donors, recipient organizations, and collection teams about new donations and collection updates.
- To visualize food waste levels, consumption trends, and bin sensor status.
- To generate usage summaries and collection reports.
- To store reliable historical waste records and system activity logs.

2.2 Functional Requirements

Functional requirements describe the operations the system must perform.

ID	Requirement	Description
FR1	User Authentication	Allow authorized users to sign in securely.
FR2	Sensor Data Collection	Collect waste bin level, flow, and collection status readings.
FR3	Reading Validation	Validate bin sensor values and identify missing readings.
FR4	Leakage Detection	Detect unusual flow and possible food waste bin overflow.
FR5	Overflow Monitoring	Identify high waste bin levels and bin overflow risks.
FR6	Dashboard Visualization	Display live levels, usage graphs, and device status.
FR7	Alert Management	Send notifications for waste accumulation, bin overflow, or low food waste.
FR8	Report Generation	Generate food waste and collection reports.
FR9	Activity Logging	Record device events, alerts, and user actions.

2.3 Non-Functional Requirements

Non-functional requirements describe the quality and performance expected from the system.

ID	Requirement	Description
NFR1	Scalability	Support more collection points and bin bin sensors and waste record records.
NFR2	Security	Protect user information and connected devices.
NFR3	Performance	Update dashboards and process alerts efficiently.
NFR4	Reliability	Handle device outages and preserve readings.
NFR5	Maintainability	Allow modules and bin sensor integrations to be updated.
NFR6	Availability	Provide dependable monitoring access.
NFR7	Usability	Offer a clear interface for donors and users and managers.
NFR8	Data Integrity	Maintain accurate, timestamped bin sensor records.

2.4 Key Quality Attributes

The architecture must consider six important quality attributes:

- Scalability: Handles increasing numbers of collection points and bin bin sensors, buildings, and readings.
- Security: Protects user accounts, device communication, and food waste data.
- Performance: Processes waste reports and bin-status updates and sends alerts promptly.
- Reliability: Maintains dependable monitoring despite temporary device failures.
- Maintainability: Enables individual modules and collection points and bin bin sensors to be updated.
- Availability: Keeps monitoring and alerts accessible when needed.

3. Selection of Suitable Software Architecture

3.1 Selected Architecture: Hybrid Architecture

A Hybrid Architecture combining Layered Architecture and Microservices is suitable for the Online Food Waste Management System. It separates the dashboard, application logic, waste processing, analytics, and database into clear layers, while allowing alerting and waste data ingestion to scale as separate services.

3.2 Architecture Components

The proposed architecture consists of the following layers:

Layer	Components	Purpose
Presentation Layer	Web Dashboard, Usage Charts	Shows live food waste status and notifications.
Application Layer	Authentication, Reports, Alert Settings	Manages user requests and application operations.
Waste Processing Layer	Waste Data Ingestion, Validation, Monitoring	Processes incoming IoT bin sensor waste records.
Analytics and Alert Layer	Threshold Rules, Usage Analysis, Notifications	Detects anomalies and issues timely alerts.
Data Layer	Database, Sensor History, Event Logs	Stores waste records and collection records.
External Interface Layer	IoT Sensors, Gateways, Notification APIs	Connects field devices and external services.

3.3 Justification for Selecting Hybrid Architecture

The hybrid approach is suitable for the following reasons:

1. Modularity: Separates bin sensor monitoring, analytics, alerts, and reporting.
2. Scalability: Allows waste data ingestion and alert processing to expand independently.
3. Maintainability: Enables updates to collection points and bin bin sensors, dashboards, or processing modules.
4. Security: Supports authenticated users, device validation, and secure APIs.
5. Performance: Processes continuous readings separately from dashboard requests.
6. Extensibility: Supports additional collection points and bin bin sensors, buildings, and prediction features.

3.4 Architecture Diagram

STAKEHOLDERS	SYSTEM ARCHITECTURE — DATA FLOW ↑	RESULTS
Residents • Food waste authorities • Maintenance team	① IoT SENSOR NETWORK Food waste-level collection points and bin bin sensors | Flow meters | Pressure collection points and bin bin sensors | Leak collection points and bin bin sensors	Live waste records

	▲ Sensor readings via Wi-Fi / LoRa / 4G ▲	
Device registration & access	② COMMUNICATION / API GATEWAY Secure ingestion | Device validation | Message queue	Connected devices
	▲ Validated bin sensor events ▲	
Administrator controls	③ DATA PROCESSING SERVICES Data cleaning | Tank-level monitoring | Flow/collection status analysis | Leak detection	Abnormality events
	▲ Processed metrics and events ▲	
	④ ANALYTICS & DECISION ENGINE Usage trends | Demand forecasting | Threshold rules | Alert triggers	Warnings & insights
	↕ Read / write ↕	
	⑤ DATABASE & CLOUD STORAGE Time-series waste reports and bin-status updates | User accounts | Alert history | Reports	Backup & audit logs
	↕ Secure APIs and notifications ↕	
Web / mobile users	⑥ APPLICATION & PRESENTATION Authentication | Web dashboard | Mobile view | Reports | Alert delivery	Live dashboard • SMS/email alerts • Reports

3.5 Architectural Trade-offs

A hybrid design adds service coordination, deployment effort, and communication overhead. Sensor failures and network interruptions also require buffering and recovery mechanisms. For an academic prototype, components can run as modules in one application before being separated into independent services.

4. Explanation of Components and Their Interactions

4.1 Component Description

The proposed architecture consists of the following components:

- Presentation Layer
  Provides a web dashboard for donors and users, facility managers, and administrators to view waste bin levels, consumption charts, bin sensor health, and alerts.
- Application Layer
  Handles user authentication, device registration, alert settings, food waste reports, and dashboard requests.
- Waste Processing Layer
  Receives food waste quantity, flow, and collection status readings; validates waste records and identifies unusual conditions.

- Analytics and Alert Layer
  Analyzes usage trends, checks bin overflow and waste accumulation thresholds, and generates notifications for responsible users.
- Data Layer
  Stores bin sensor waste records, waste bin information, user profiles, alert history, and collection records.
- External Interface Layer
  Connects bin collection points and bin bin sensors and collection-point inputs, network gateways, notification services, and optional weather or utility APIs.

4.2 Interaction Between Components

Sensors transmit readings through a gateway to the Waste Processing Layer. Validated data is saved in the Data Layer and evaluated by the Analytics and Alert Layer. The Application Layer provides secure access to current waste records and reports, while the Presentation Layer displays status and notifications.

4.3 System Workflow

- The authorized user signs in to the monitoring dashboard.
- The administrator registers waste bins, collection points and bin bin sensors, and alert thresholds.
- Food waste level, flow, and collection status collection points and bin bin sensors collect waste records.
- The communication gateway sends waste reports and bin-status updates to the backend.
- The processing module validates and timestamps the readings.
- The system stores waste records and evaluates waste accumulation or bin overflow rules.
- The dashboard shows current waste bin levels and consumption trends.
- When abnormal conditions occur, the system notifies the responsible team.
- Maintenance actions and important events are recorded in the system log.

5. Discussion of Quality Attributes:

5.1 Scalability

The system can support additional waste bins, buildings, and bin sensor devices. Sensor ingestion and alert services can be scaled independently when the number of readings increases.

5.2 Security

The platform uses secure authentication, role-based permissions, encrypted communication, and controlled device access. Activity logs support accountability and traceability.

5.3 Performance

Efficient ingestion, indexed database queries, and asynchronous alerts help process frequent waste reports and bin-status updates without delaying dashboard updates.

5.4 Reliability

Validation rules, retry mechanisms, local buffering, and backups help prevent lost readings during network failures. Fault alerts identify disconnected or malfunctioning collection points and bin bin sensors.

5.5 Maintainability

The layered, modular design separates dashboards, application functions, waste data ingestion, analytics, and storage so individual parts can be tested or replaced with limited impact.

5.6 Availability

Service monitoring, database backups, and recovery procedures improve availability. Redundant communication paths or deployments can be added for larger installations.

6. Justification of Architectural Decisions

6.1 Architectural Justification

The proposed Online Food Waste Management System uses a Hybrid Architecture combining Layered Architecture and Microservices. The approach organizes responsibilities into layers while allowing high-volume waste processing and alerting to operate independently when needed.

The architecture is suitable because:

- Modularity: Each layer performs clearly defined food waste-monitoring functions.
- Scalability: Sensor ingestion and alerts can support more devices independently.
- Security: Authentication and encrypted communication protect access and data.
- Maintainability: Sensor and application modules can be updated separately.
- Extensibility: Additional bin sensor types and forecasting services can be integrated.

6.2 Architectural Trade-offs

Although the hybrid architecture offers several benefits, it also has some limitations:

- Implementation complexity: Multiple services require configuration and coordination.
- Communication overhead: Network communication can delay incoming readings.
- Deployment effort: Gateways and distributed services need monitoring and support.
- Security management: Devices, APIs, and databases must be protected.

For the initial academic prototype, the solution can use one modular backend connected to a small bin sensor network. Services may be separated as deployment size and waste record volume grow.

6.3 Future Enhancements

The architecture can be improved in the future by introducing:

1. AI-based prediction of food waste demand and abnormal consumption.
2. Integration with additional food waste quality and collection status collection points and bin bin sensors.
3. Automatic pump control with appropriate safety safeguards.
4. Interactive maps showing waste bins, pipelines, and waste accumulation alerts.
5. Cloud-based monitoring for multiple buildings and facilities.
6. Improved maintenance scheduling and long-term food waste forecasting.

7. Conclusion

The Online Food Waste Management System uses a hybrid architecture combining Layered Architecture and Microservices to enable reliable monitoring of food waste levels, consumption, flow, and collection status. It integrates IoT bin sensor collection, processing, analytics, alerting, secure storage, and dashboard visualization.

The proposed design improves scalability, security, maintainability, and timely detection of waste accumulation or bin overflow. It helps donors and users and facility managers respond to problems and make informed food waste management decisions. The architecture provides a flexible foundation for future automation and expansion.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software	Selects the most appropriate	Selects a suitable architecture with	Architecture is partially	Inappropriate architecture

Architecture (25%)	architectural style with strong technical justification aligned to system requirements.	reasonable justification.	suitable with limited justification.	selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25